

Sicurezza

CdL in Informatica – L31

Crittografia Asimmetrica

Prof. Emiliano Casalicchio

Department of Computer Science

Sapienza University of Rome



SAPIENZA
UNIVERSITÀ DI ROMA



Chapter 21 (CS)

Hash functions and Asymmetric Encryption



Topics

- Hash function & Secure hash functions
 - SHA, HMAC
- Authenticated encryption
 - OCB - AE
- Public-key encryption
 - RSA
 - DE



Message Authentication (Recap)

	Protects against active attacks
	Verifies received message is authentic • Contents have not been altered • From authentic source • Timely and in correct sequence
	Can use conventional encryption • Only sender and receiver share a key

in the ECB mode of encryption, if an attacker reorders the blocks of ciphertext, then each block will still decrypt successfully.

However, the reordering may alter the meaning of the overall data sequence.

Although sequence numbers may be used at some level (e.g., each IP packet), it is typically not the case that a separate sequence number will be associated with each *b-bit block of plaintext*. Thus, *block reordering is a threat*.



Message Authentication Without Confidentiality

- Typically message authentication is provided as a separate function from message encryption (confidentiality)
- Situations in which message authentication without confidentiality may be preferable include:
 - the same message is broadcast to a number of destinations
 - one side has a heavy load and cannot afford the time to decrypt all incoming messages
 - Authentication of a computer program in plaintext is an attractive service



Message Authentication Code (MAC)

Use of a secret key to generate a small block of data, known as a message authentication code, that is appended to the message

DES / AES
some of the bits of ciphertext are used as the code

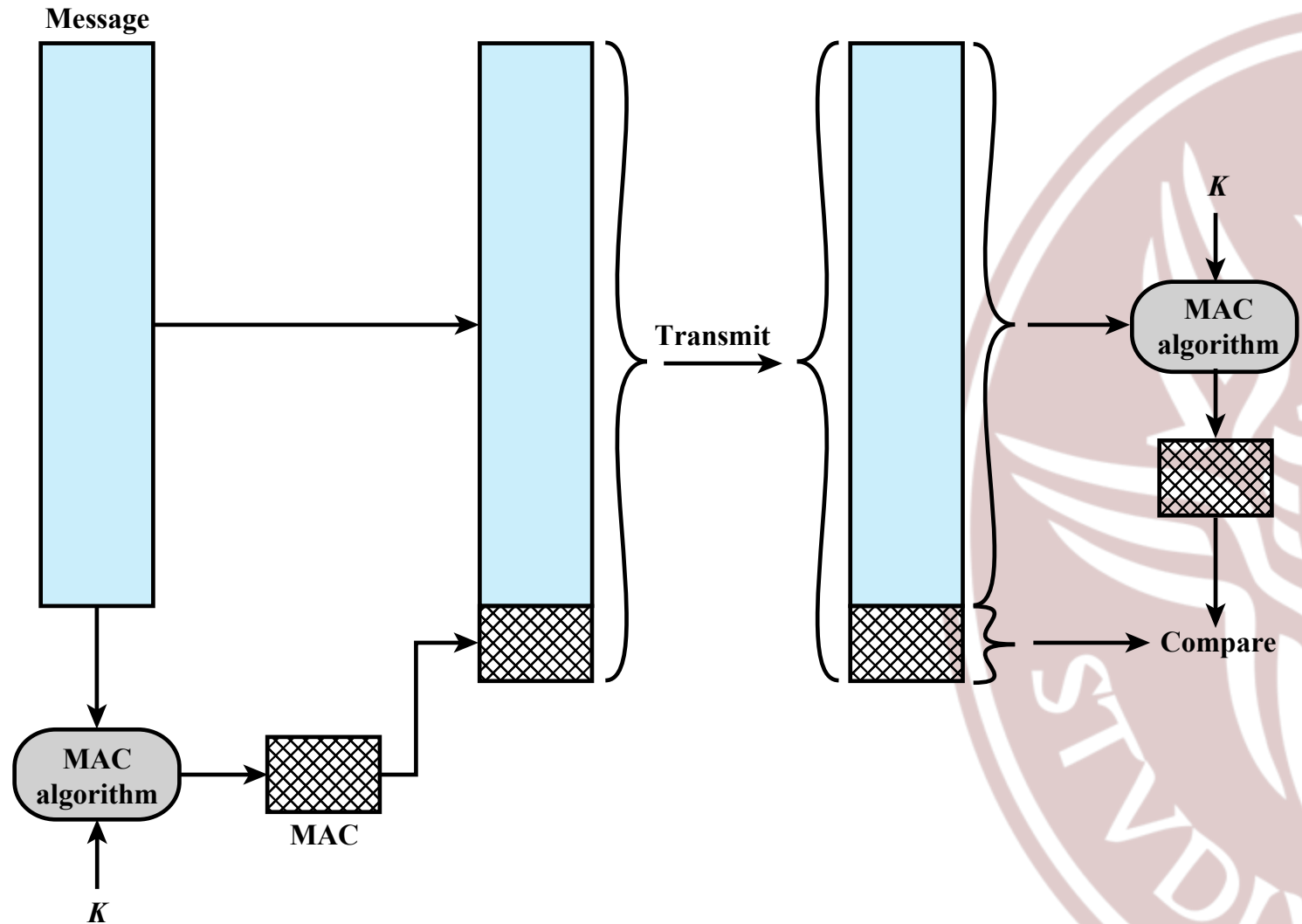


Figure 2.3 Message Authentication Using a Message Authentication Code (MAC).

Message Authentication Code (MAC) (cont'd)

If we assume that

- only the receiver and the sender know the identity of the secret key
- and if the received code matches the calculated code

then

- **The receiver is assured that the message has not been altered.**
 - If an attacker alters the message but does not alter the code, then the receiver's calculation of the code will differ from the received code. Because the attacker is assumed not to know the secret key, the attacker cannot alter the code to correspond to the alterations in the message.
- **The receiver is assured that the message is from the alleged sender.**
 - Because no one else knows the secret key, no one else could prepare a message with a proper code.
- **The receiver can be assured of the proper sequence**
 - if the message includes a sequence number (such as is used with X.25, HDLC, and TCP), because an attacker cannot successfully alter the sequence number.

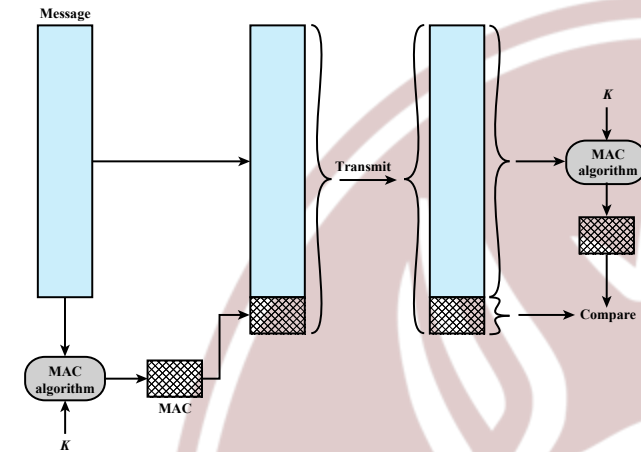
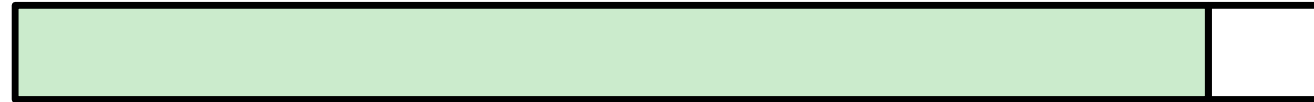


Figure 2.3 Message Authentication Using a Message Authentication Code (MAC).

One-way hash function



The message is padded out to an integer multiple of some fixed length (e.g., 1024 bits) and the padding includes:

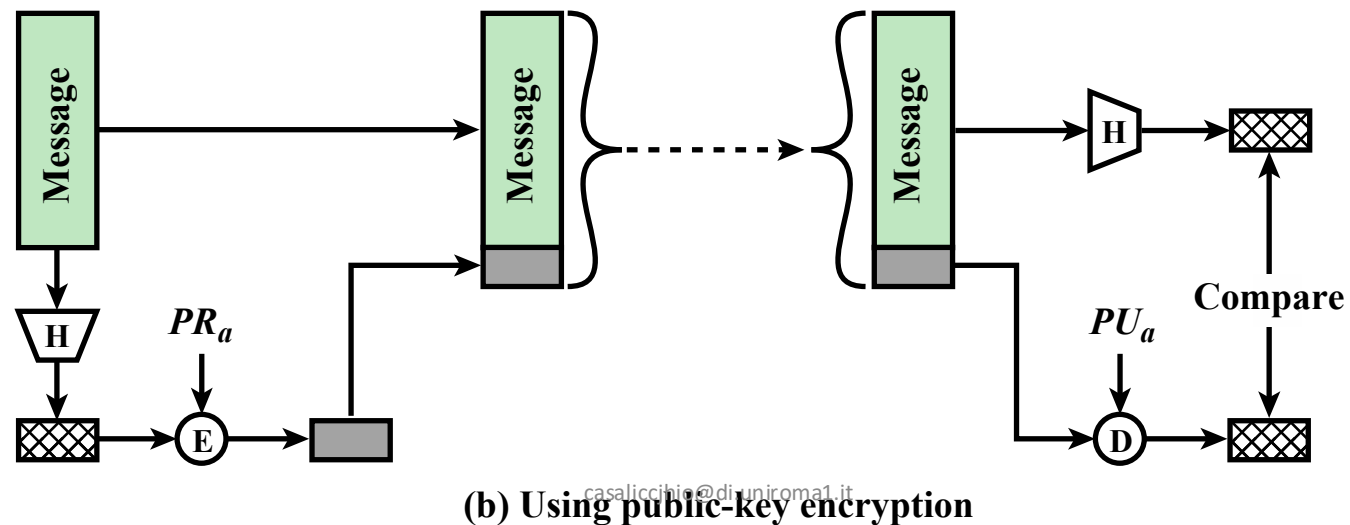
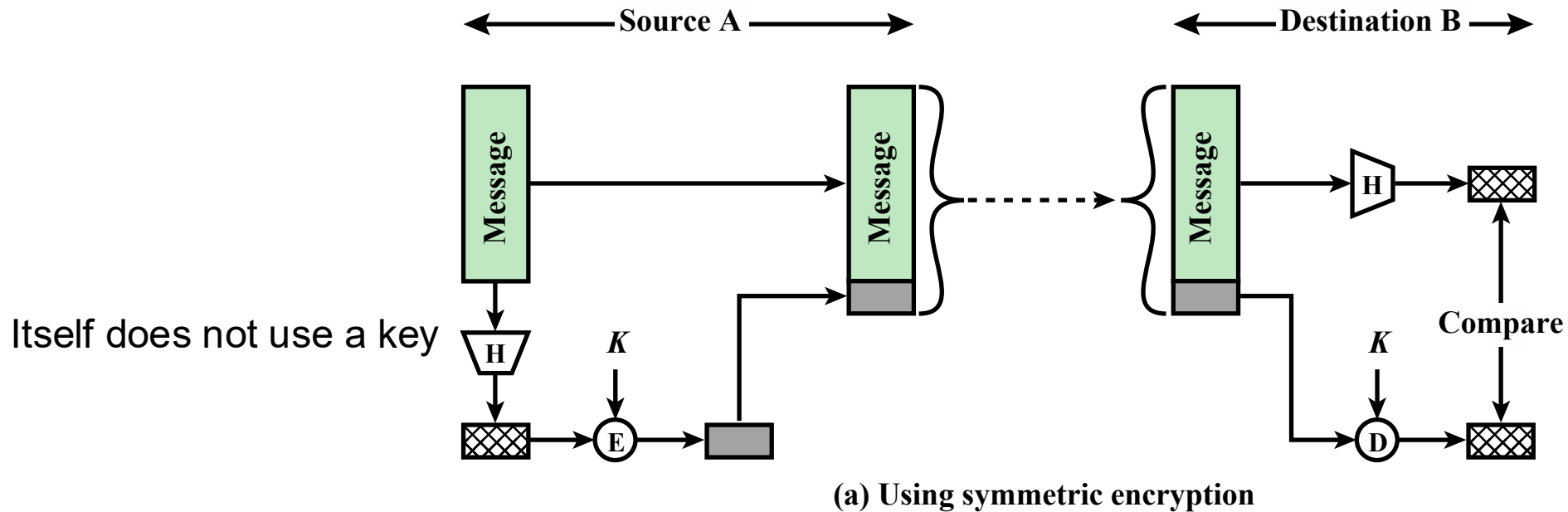
- **the value of the length of the original message in bits**

Is a security measure. Why?

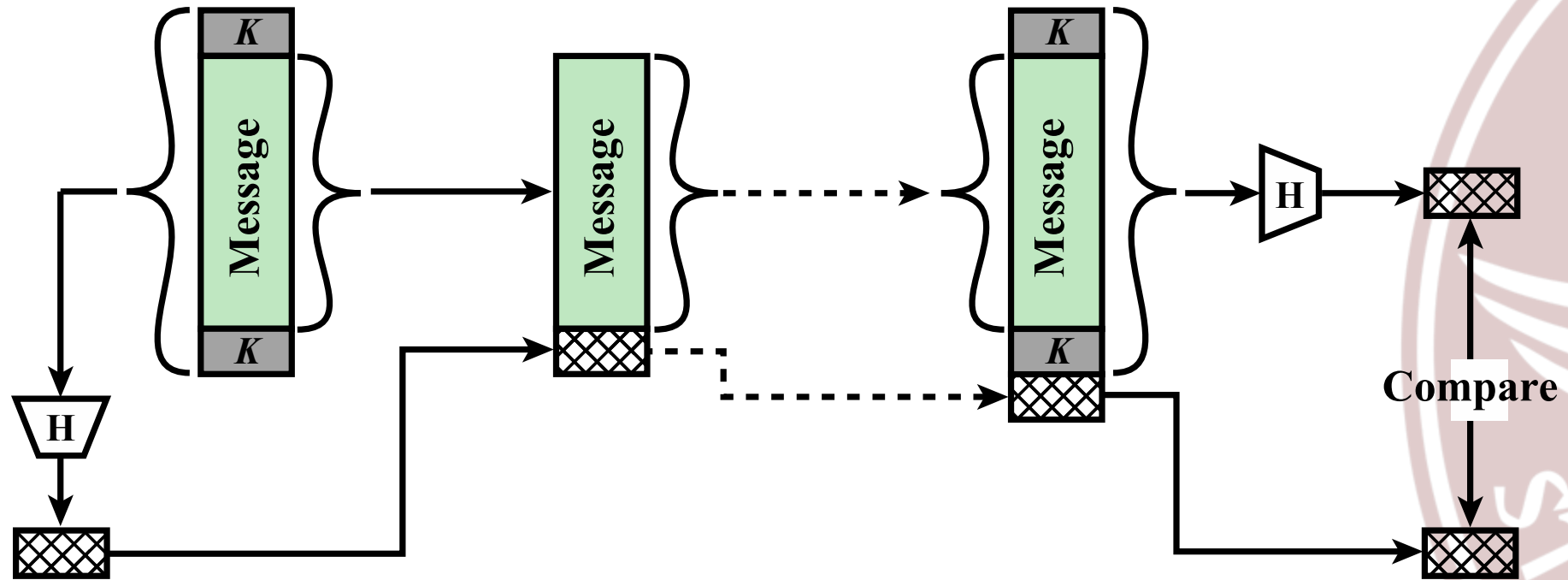


$H(M)$ message digest

Ways in which the message can be authenticated using a hash code



Ways in which the message can be authenticated using a hash code – No encryption.



(c) Using secret value

Figure 2.5 Message Authentication Using a One-Way Hash Function.

To be useful for message authentication, a hash function H must have the following properties:

1) Can be applied to a block of data of any size

2) H produces a fixed-length output

3) $H(x)$ is relatively easy to compute for any given x

4) One-way or pre-image resistant

- Computationally infeasible to find x such that $H(x) = h$

5) Computationally infeasible to find for any $x, y \neq x$ such that $H(y) = H(x)$

6) Collision resistant or strong collision resistance

- Computationally infeasible to find any pair (x,y) such that $H(x) = H(y)$



- **The first three** properties are requirements for the practical application of a hash function to message authentication
- **The fourth property** say It is easy to generate a code given a message, but virtually impossible to generate a message given a code.
 - is important if the authentication technique involves the use of a secret value
 - if the hash function is not one way, an attacker can easily discover the secret value:
 - If the attacker can observe or intercept a transmission, the attacker obtains the message M and the hash code $MD_M = H(K \parallel M \parallel K)$. The attacker then inverts the hash function to obtain $K \parallel M \parallel K = H^{-1}(MD_M)$. Because the attacker now has both M and $K \parallel M \parallel K$, it is a trivial matter to recover K .
- **The fifth property** guarantees that it is impossible to find an alternative message with the same hash value as a given message. This prevents forgery when an encrypted hash code is used
 - If this property were not true, an attacker would be capable of the following sequence:
 - First, observe or intercept a message plus its encrypted hash code;
 - second, generate an unencrypted hash code from the message;
 - third, generate an alternate message with the same hash code.
- **Weak Hash function** if 1 – 5 are satisfied
- **Strong Hash Function** if 1 – 6 are satisfied



Hash functions for message auth & integrity

- MAC (Message Authentication Code)
- Cryptographic hash function
- One-way hash function



Secure Hash Function

- The one-way hash function, or secure hash function, is important not only in message authentication but also in digital signatures
- All hash functions operate using the following general principles:
 - The input (message, file, etc.) is viewed as a sequence of n -bit blocks
 - The input is processed one block at a time in an iterative fashion to produce an n -bit hash function



Simplest hash function

Bitwise XOR

	Bit 1	Bit 2	• • •	Bit n
Block 1	b_{11}	b_{21}		b_{n1}
Block 2	b_{12}	b_{22}		b_{n2}
	•	•	•	•
	•	•	•	•
	•	•	•	•
Block m	b_{1m}	b_{2m}		b_{nm}
Hash code	C_1	C_2		C_n

- produces a simple parity for each bit position
- is known as a **longitudinal redundancy check**.
- effective for random data as a data integrity check



Improvement of simplest hash function

- To perform 1-bit circular switch of the hash value after each block is processed

Procedure

1. Initialization

- set the hash n-bits to zero

2. Process each successive n-bit block of data as follow

- Rotate the current hash value on the left of 1 bit
 - XOR the block into the hash value
 - Repeat 2 for each block
- This has the effect of “randomizing” the input more completely and overcoming any regularities that appear in the input.



The Simple XOR or RXOR schema is not strong collision resistant if the encrypted hash code is used with plaintext

- it is an easy to produce a new message that yields the desired hash code:
 - prepare the desired alternate message
 - append an n-bit block that forces the new message plus block to yield the desired hash code
- Hash code (digest) encrypted with the plaintext eliminate this issue ... but
- ... what could be the vulnerability?



Simple XOR or (RXOR) with message and diggest encryption

- Let us assume a technique that uses the **simple XOR** applied to 64-bit blocks of the message and then an encryption of the entire message that uses the cipher block chaining (CBC) mode
- The scheme is as follows: Given a message consisting of a sequence of 64-bit blocks $X_1, X_2, \dots, X_N$, define the hash code C as the block-by-block XOR of all blocks and append the hash code as the final block:
 - $C = X_{N+1} = X_1 \text{ xor } X_2 \dots \text{ xor } \dots X_N$
- Next, encrypt the entire message plus hash code C , using CBC mode to produce the encrypted message
 - $Y_1, Y_2, \dots, Y_{N+1}$



At decryption we have

- $X_1 = IV \text{ XOR } D(K, Y_1)$
- $X_i = Y_{i-1} \text{ XOR } D(K, Y_i)$
- And the hash code $X_{N+1} = Y_N \text{ XOR } D(K, Y_{N+1})$
- $X_{N+1} = X_1 \text{ XOR } X_2 \dots \text{ XOR } X_N =$
 $[IV \text{ XOR } D(K, Y_1)] \text{ XOR } [Y_1 \text{ XOR } D(K, Y_2)] \dots \text{ XOR}$
 $[Y_{N-1} \text{ XOR } D(K, Y_N)]$
- Rearranging the terms in any order does not change the result
- **An attack that changes the order of the messages is not detected**



Secure Hash Algorithm (SHA)

- SHA was originally developed by NIST
- Published as FIPS 180 in 1993
- Was revised in 1995 as **SHA-1**
 - Produces 160-bit hash values
- NIST issued revised FIPS 180-2 in 2002 (**SHA-2**)
 - Adds 3 additional versions of SHA
 - SHA-256, SHA-384, SHA-512
 - With 256/384/512-bit hash values
 - Same basic structure as SHA-1 but greater security
- The most recent version is FIPS 180-4 (2015) which added two variants of SHA-512 with 224-bit and 256-bit hash sizes (SHA-512 more efficient than SHA-256)

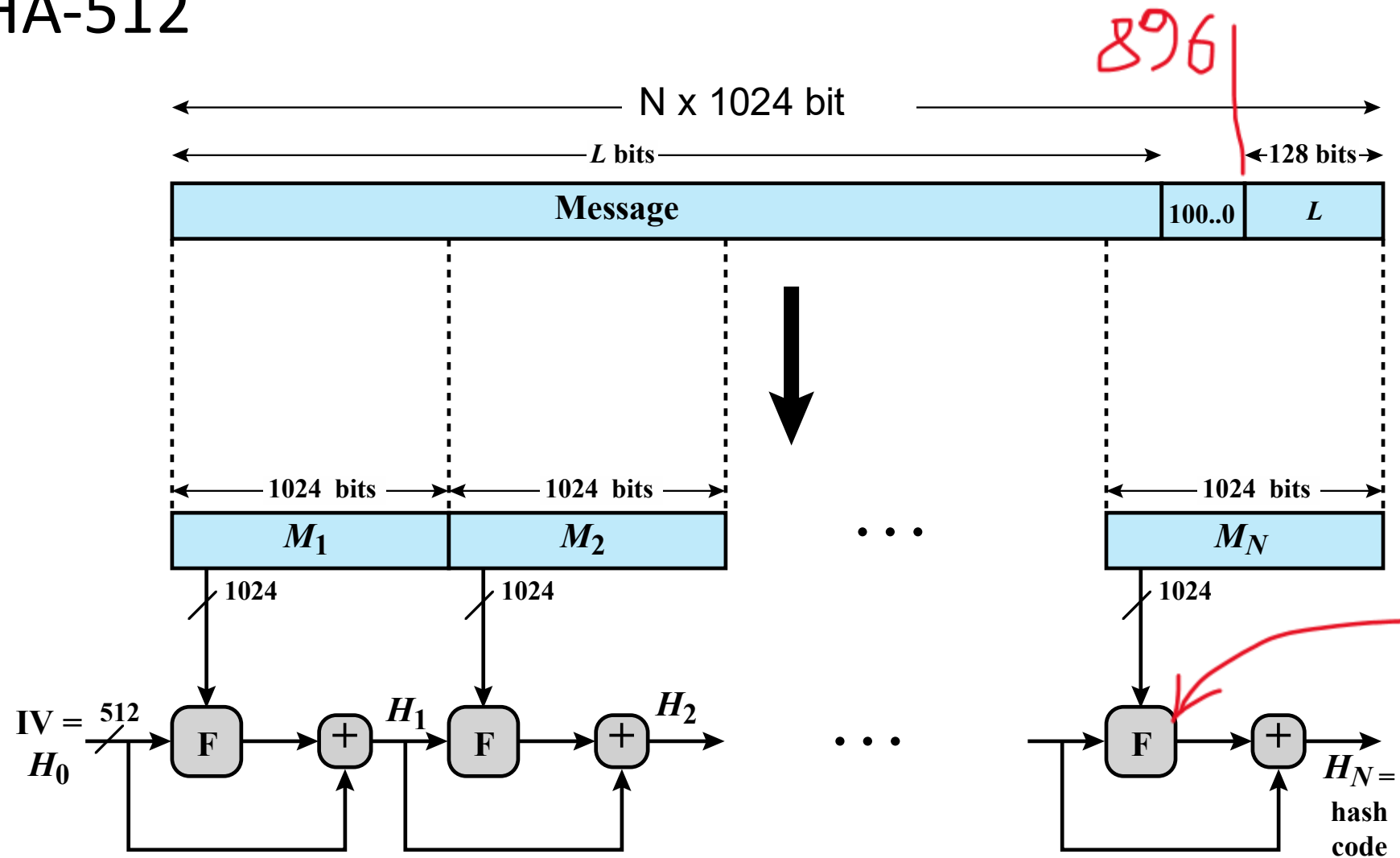


Comparison of SHA parameters

- All sizes are measured in bits
- Security refers to the fact that a birthday attack on a message digest of size n produces a collision with a work factor of approximately $2^{n/2}$.

	SHA-1	SHA-224	SHA-256	SHA-384	SHA-512	SHA-512/224	SHA-512/256
Message size	$< 2^{64}$	$< 2^{64}$	$< 2^{64}$	$< 2^{128}$	$< 2^{128}$	$< 2^{128}$	$< 2^{128}$
Word size	32	32	32	64	64	64	64
Block size	512	512	512	1024	1024	1024	1024
Message digest size	160	224	256	384	512	224	256
Number of steps	80	64	64	80	80	80	80
Security	80	112	128	192	256	112	128

SHA-512

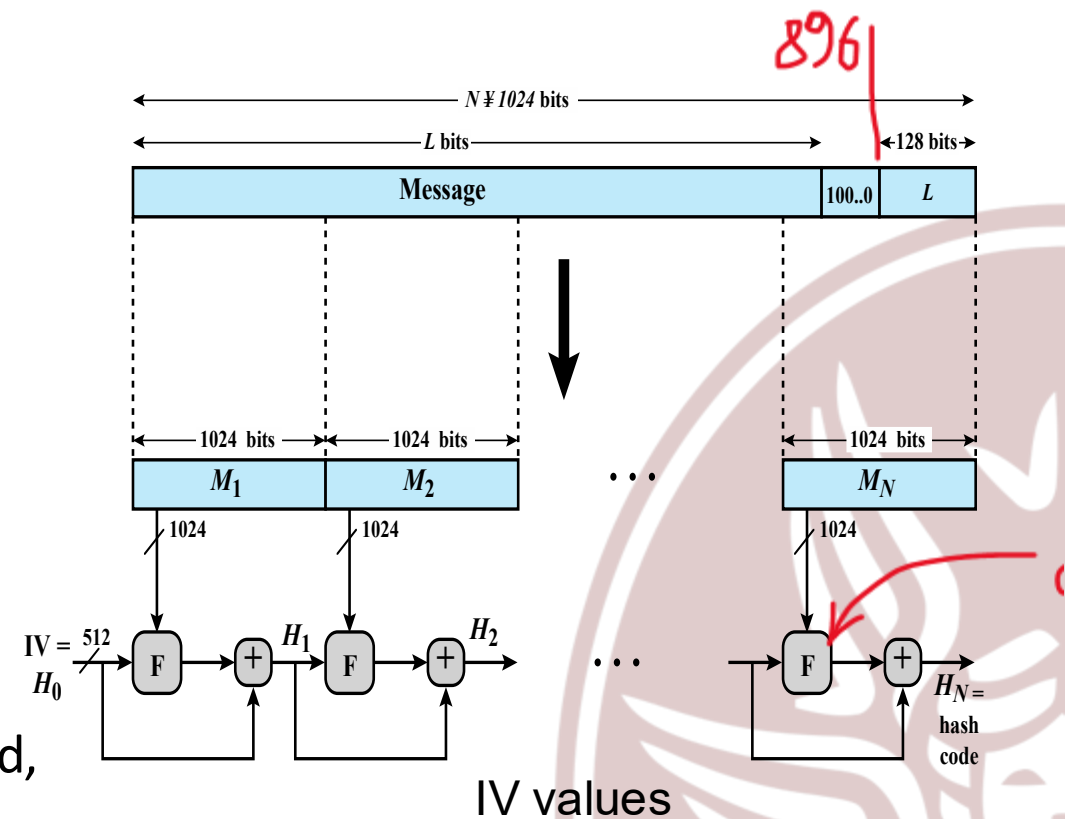


$+$ = word-by-word addition mod 2^{64}



SHA-512

1. **Append padding bits:** so that message length is congruent to 896 modulo 1024. Padding consists of a single 1-bit followed by a number of 0-bits.
2. **Append length:** as a block of 128 bits being an unsigned 128-bit integer length of the original message
3. **Initialize hash buffer:** A 512-bit buffer is used to hold intermediate and final results of the hash function. The buffer can be represented as eight 64-bit registers (a, b, c, d, e, f, g, h)
4. **Process the message in 1024-bit (128-byte) blocks:** using the module F that consists of 80 rounds
5. **Output.** After all N 1024-bit blocks have been processed, the output from the N th stage is the 512-bit message digest.



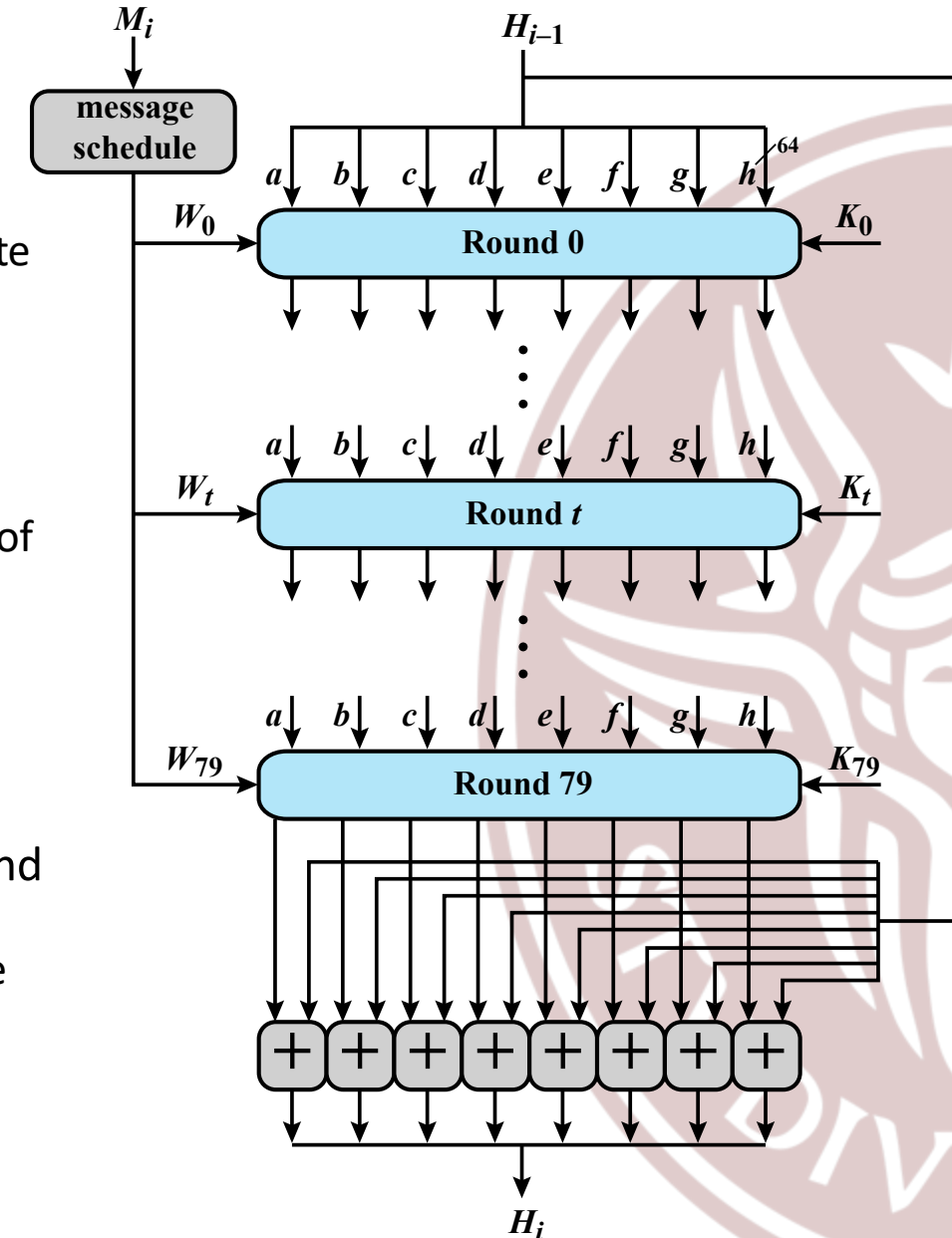
IV values

a = 6A09E667F3BCC908
b = BB67AE8584CAA73B
c = 3C6EF372FE94F82B
d = A54FF53A5F1D36F1

e = 510E527FADE682D1
f = 9B05688C2B3E6C1F
g = 1F83D9ABFB41BD6B
h = 5BE0CD19137E2179

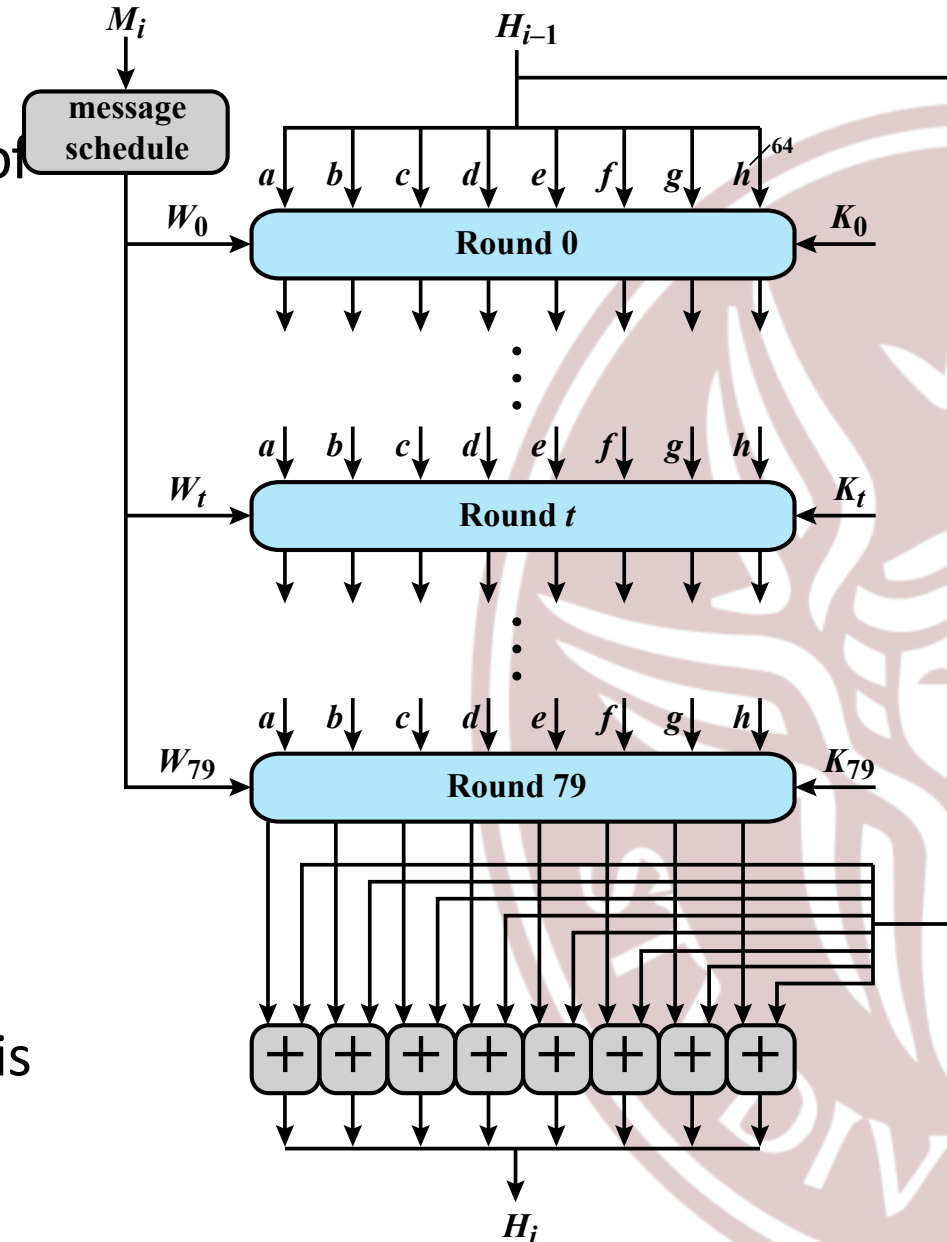
F: core of SHA-512

- Each round takes as input the 512-bit buffer value abcdefgh, and updates the contents of the buffer.
 - At input to the first round, the buffer has the value of the intermediate hash value, H_{i-1}
- Each round t makes use of a 64-bit value W_t , derived from the current 1024-bit block being processed (M_i)
- Each round also makes use of an additive constant K_t
 - K_t represent the first 64 bits of the fractional parts of the cube roots of the first 80 prime numbers
 - K_t provides a “randomized” set of 64-bit patterns, which should eliminate any regularities in the input data
- Round t operation consist of circular shifts, and primitive Boolean functions based on AND, OR, NOT, and XOR
- The output of the 80th round is added to the input to the 1st round (H_{i-1}) to produce H_i .
 - The addition is done independently for each of the eight words in the buffer with each of the corresponding words in H_{i-1} , using addition modulo 2^{64}



SHA-512 F properties

- Every bit of the hash code is a function of every bit of the input
 - The complex repetition of the basic function F produces **results that are well mixed**;
 - it is unlikely that two messages chosen at random, even if they exhibit similar regularities, will have the same hash code.
- 
- The difficulty of coming up with two messages having the same message digest is on the order of 2^{256} operations
 - The difficulty of finding a message with a given digest is on the order of 2^{512} operations



SHA-3

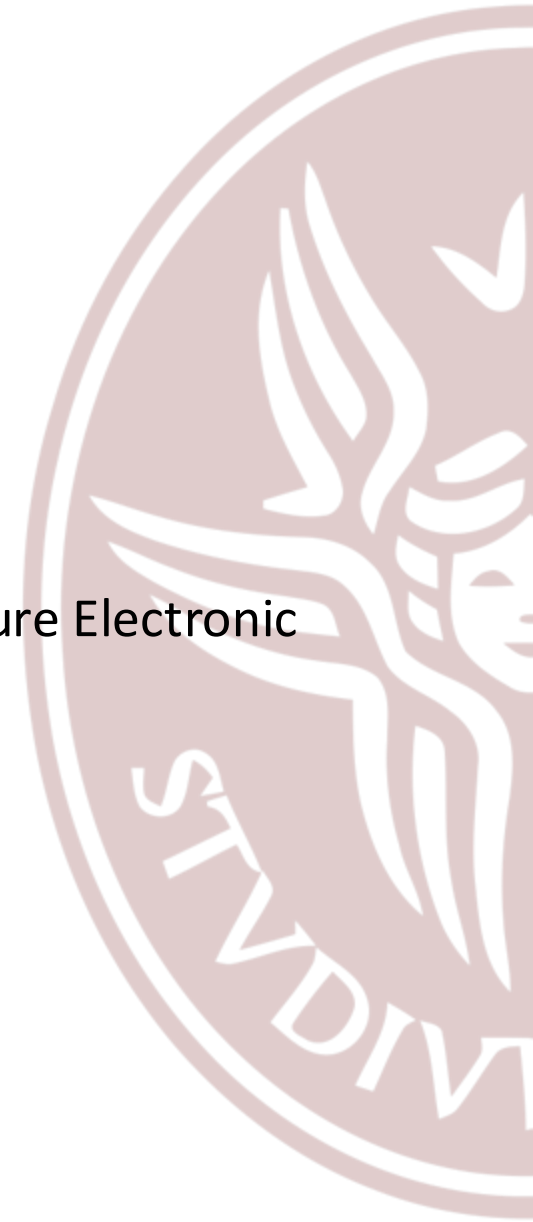
- SHA-2 shares same structure and mathematical operations as its predecessors and causes concern
- Due to time required to replace SHA-2 should it become vulnerable, NIST announced in 2007 a competition to produce SHA-3

Requirements:

- Must support hash value lengths of 224, 256, 384, and 512 bits
- Algorithm must process small blocks at a time instead of requiring the entire message to be buffered in memory before processing it

HMAC: Keyed-Hashing for Message Authentication

- Interest in developing a MAC derived from a cryptographic hash code
 - Cryptographic hash functions generally execute faster
 - Library code is widely available
 - **SHA-1 was not designed for use as a MAC because it does not rely on a secret key**
- HMAC Has been chosen as the mandatory-to-implement MAC for IP security
- Used in other Internet protocols such as Transport Layer Security (TLS) and Secure Electronic Transaction (SET)



HMAC Design Objectives

- **To use, without modifications, available hash functions**
 - in particular, hash functions that perform well in software, and for which code is freely and widely available
- **To allow for easy replaceability of the embedded hash function**
 - in case faster or more secure hash functions are found or required
- **To preserve the original performance of the hash function**
 - without incurring a significant degradation
- **To use and handle keys in a simple way**
- **To have a well-understood cryptographic analysis of the strength of the authentication mechanism**
 - based on reasonable assumptions on the embedded hash function



HMAC Algorithm

H = embedded hash function (e.g., SHA)

M = message input to HMAC (including the padding specified in the embedded hash function)

Y_i = i th block of M , $0 \leq i \leq (L - 1)$

L = number of blocks in M

b = number of bits in a block

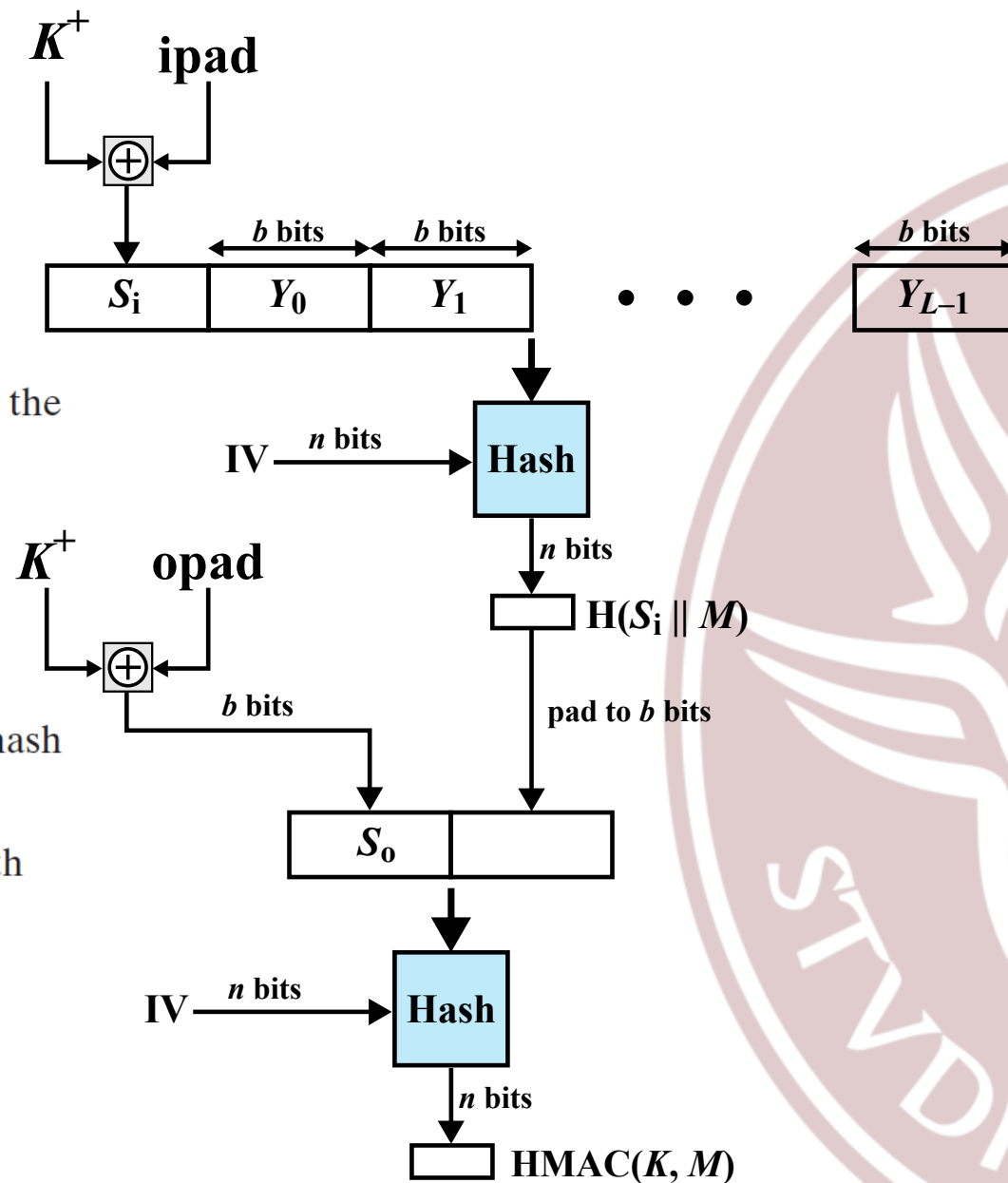
n = length of hash code produced by embedded hash function

K = secret key; if key length is greater than b , the key is input to the hash function to produce an n -bit key; recommended length is $\geq n$

K^+ = K padded with zeros on the left so that the result is b bits in length

ipad = 00110110 (36 in hexadecimal) repeated $b/8$ times

opad = 01011100 (5C in hexadecimal) repeated $b/8$ times



HMAC Algorithm

1. Append zeros to the left end of K to create a b -bit string K^+ (e.g., if K is of length 160 bits and $b = 512$, then K will be appended with 44 zero bytes 0x00).
2. XOR (bitwise) K^+ with ipad to produce the b -bit block S_i .
 - flips one-half of the bits of K
3. Append M to S_i .
4. Apply H to the stream generated in step 3.
5. XOR K^+ with opad to produce the b -bit block S_o .
 - flips one-half of the bits of K (different bits from step 2)
6. Append the hash result from step 4 to S_o .
7. Apply H to the stream generated in step 6 and output the result.

By passing S_i and S_o through the hash algorithm, we have pseudorandomly generated two keys from K .

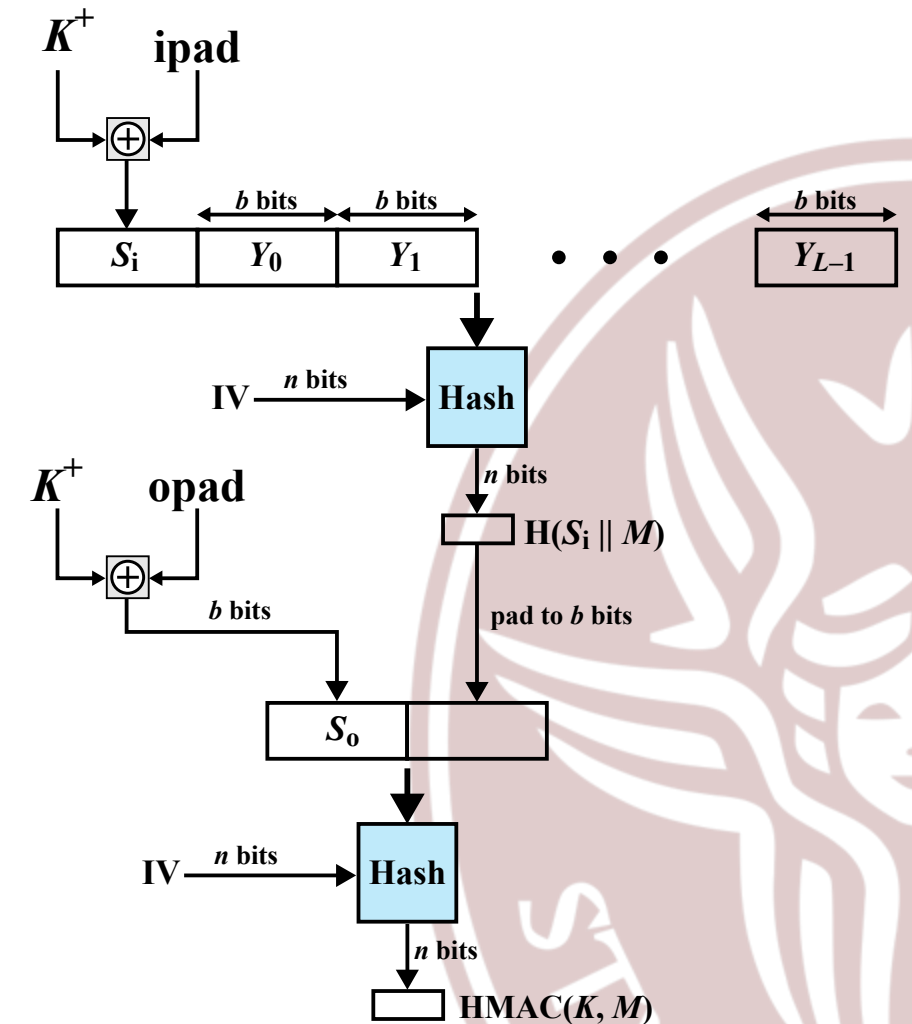
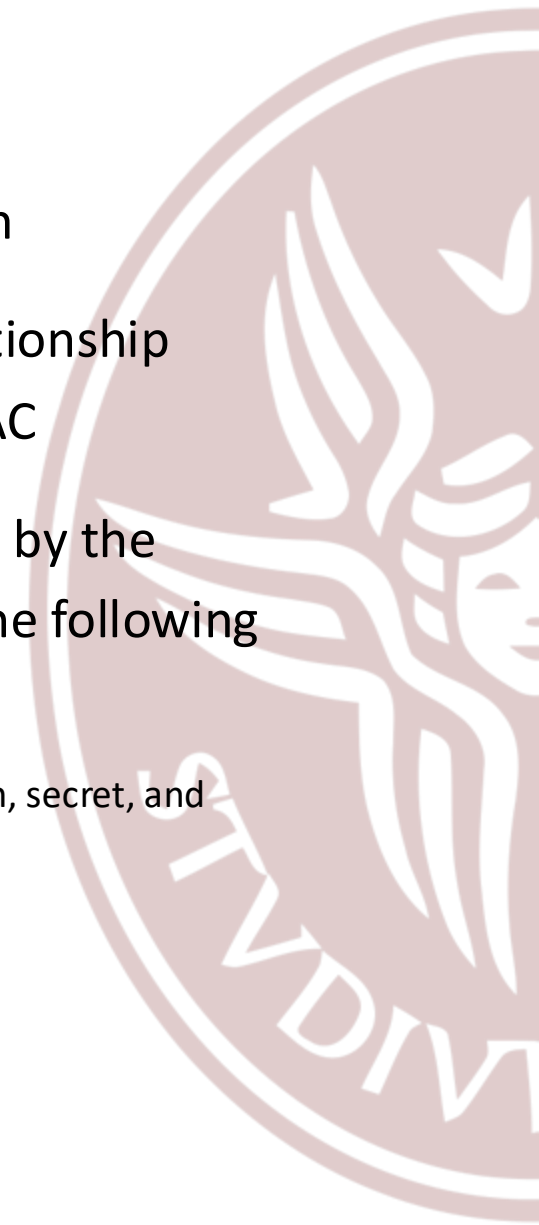


Figure 21.4 HMAC Structure

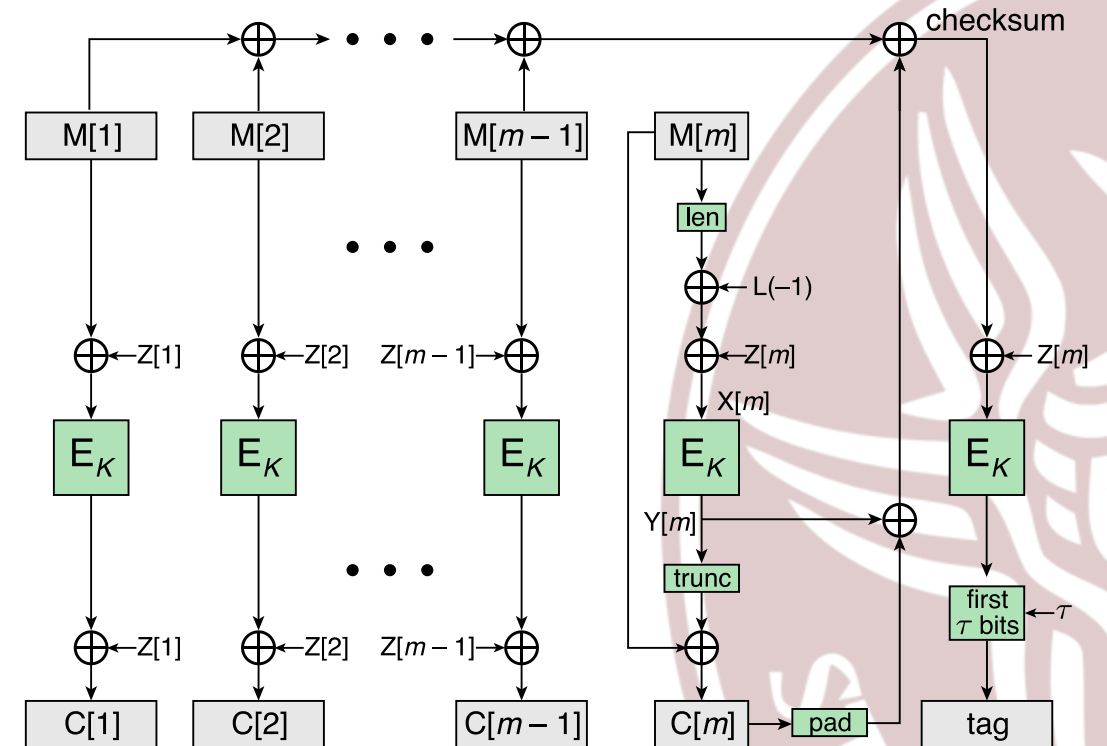
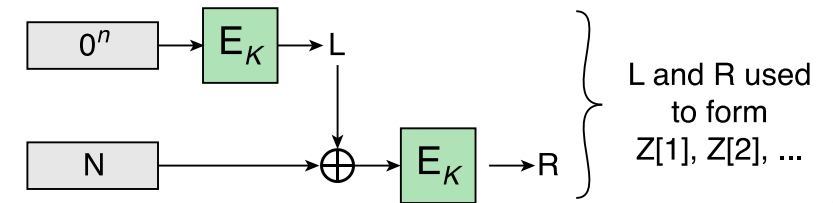
Security of HMAC

- Security depends on the cryptographic strength of the underlying hash function
- The appeal of HMAC is that its designers have been able to prove an exact relationship between the strength of the embedded hash function and the strength of HMAC
- For a given level of effort on messages generated by a legitimate user and seen by the attacker, the probability of successful attack on HMAC is equivalent to one of the following attacks on the embedded hash function:
 - The attacker is able to compute an output of the compression function even with an IV that is random, secret, and unknown to the attacker
 - The attacker finds collisions in the hash function even when the IV is random and secret



Authenticated encryption (AE)

- Encryption systems that protect confidentiality and authenticity (integrity)
 - message encryption + message authentication
- Implemented using a block cipher mode
- OCB AE (2014)
 - approved as an authenticated encryption technique in the **IEEE 802.11** wireless LAN standard and included in **MiniSec**, the open-source IoT security module
- OCB similar to ECB
 - ECB weak because same plaintext block produce same ciphertext (if the same key is applied) – not secure for lengthy messages
 - OCB introduce Z, and each block $M[i]$ is XORed with a different value of $Z[i]$
 - $C[i] = E_K(M[i] \text{ XOR } Z[i]) \text{ XOR } Z[i]$



n = block length in bits

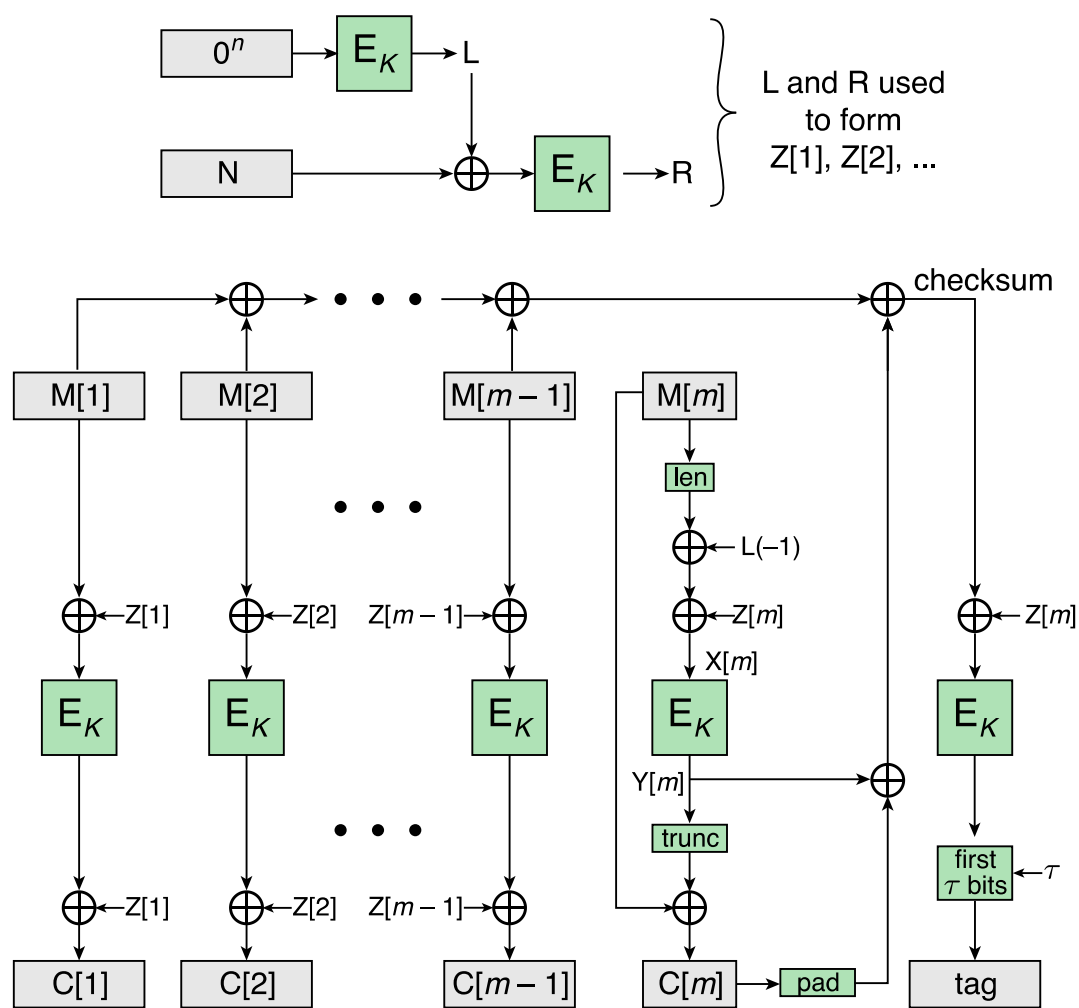
N = nonce

$\text{len}(M[m])$ = length of $M[m]$ represented as an n -bit integer

$\text{trunc}(Y[m])$ = deletes least significant bits so that result is same length as $M[m]$

pad = pad with least significant 0 bits to length n

τ = length of authentication tag



n = block length in bits

N = nonce

$\text{len}(M[m])$ = length of $M[m]$ represented as an n -bit integer

$\text{trunc}(Y[m])$ = deletes least significant bits so that result is same length as $M[m]$

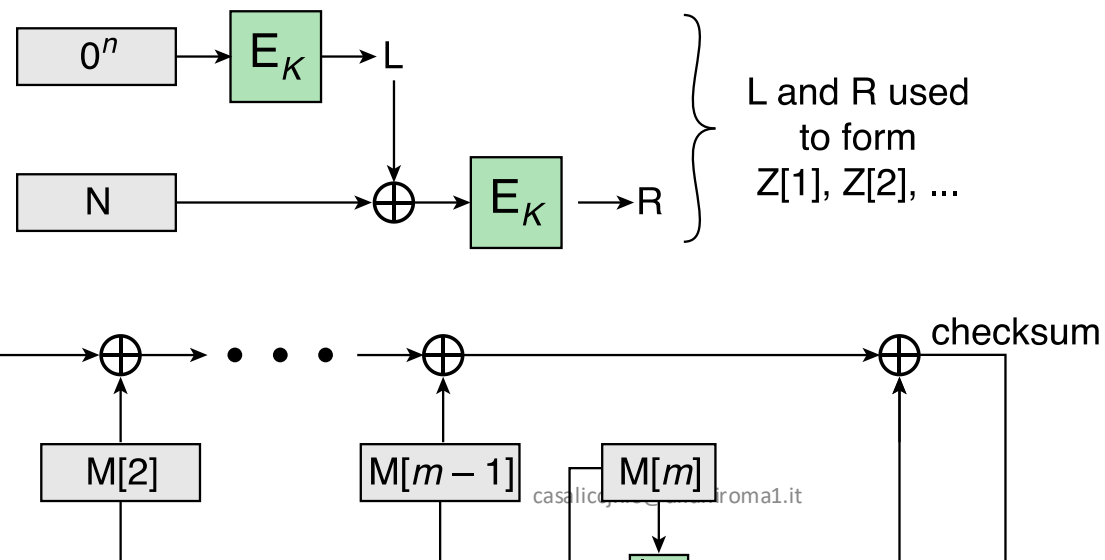
pad = pad with least significant 0 bits to length n

τ = length of authentication tag

- AES used as the encryption algorithm
- M divided into n -bit blocks ($n=128$)
 - $\text{len}(M[m]) \leq n$ bit
- The total number of blocks is
 - $m = \text{ceil}(\text{len}(M)/n)$
- N , an arbitrary n -bit value, called the **nonce** is chosen
 - if multiple messages are encrypted with the same key, a different nonce must be used each time such that each nonce is only used once.
 - each different value of N will produce a different set of $Z[i]$.

Computation of $Z[i]$

- $L[0] = L = E_k(0^n)$
- $R = E_k(N \text{ XOR } L)$
- $L[i] = 2 \cdot L[i-1]$; $1 \leq i \leq m$; where $\cdot$ is the multiplication over the finite field
- $Z[1] = L \text{ XOR } R$
- $Z[i] = Z[i-1] \text{ XOR } L[\text{ntz}(i)]$; $2 \leq i \leq m$; $\text{ntz}(i)$ is the num of less significant zeros in i



Computation of C[m], Checksum and Tag

- $\text{len}(M[m]) \leq n$ bit
- $C[m]$ computed differently from $C[i=\{1, m-1\}]$
- $C[m]0^*$ is $C[m]$ padded with least significant bit to length of n
- $L[-1] = L \cdot 2^{-1}$ (or $L/2$ over the finite field)
- The checksum is function of M and K
- Finally a Tag of length ***tau*** bit is produced
- ***tau*** determines the level of authentication
 - greater is ***tau*** stronger is the auth

```

algorithm OCB-EncryptK(N, M)
Partition M into M[1]...M[m]
L ← L(0) ← EK(0n)
R ← EK(N ⊕ L)
for i ← 1 to m do L(i) ← 2 · L(i − 1)
L(−1) = L · 2−1
Z[1] ← L ⊕ R
for i ← 2 to m do Z[i] ← Z[i − 1] ⊕ L(ntl(i))
for i ← 1 to m − 1 do
    C[i] ← EK(M[i] ⊕ Z[i]) ⊕ Z[i]
X[m] ← len(M[m]) ⊕ L(−1) ⊕ Z[m]
Y[m] ← EK(X[m])
C[m] ← M[m] ⊕ (first len(M[m]) bits of Y[m])
Checksum ←
    M[1] ⊕ ... ⊕ M[m − 1] ⊕ C[m]0* ⊕ Y[m]
Tag ← EK(Checksum ⊕ Z[m]) [first τ bits]
    
```

Decryption

Note:

```

algorithm OCB-EncryptK(N, M)
Partition M into M[1]...M[m]
L ← L(0) ← EK(0n)
R ← EK(N ⊕ L)

for i ← 1 to m do L(i) ← 2 · L(i − 1)
L(−1) = L · 2−1
Z[1] ← L ⊕ R
for i ← 2 to m do Z[i] ← Z[i − 1] ⊕ L(nty(i))
for i ← 1 to m − 1 do
    C[i] ← EK(M[i] ⊕ Z[i]) ⊕ Z[i]
X[m] ← len(M[m]) ⊕ L(−1) ⊕ Z[m]
Y[m] ← EK(X[m])
C[m] ← M[m] ⊕ (first len(M[m]) bits of Y[m])
Checksum ←
    M[1] ⊕ ... ⊕ M[m − 1] ⊕ C[m]0* ⊕ Y[m]
Tag ← EK(Checksum ⊕ Z[m]) [first τ bits]
    
```

```

algorithm OCB-DecryptK(N, M)
Partition M into M[1]...M[m]
L ← L(0) ← EK(0n)
R ← EK(N ⊕ L)

for i ← 1 to m do L(i) ← 2 · L(i − 1)
L(−1) = L · 2−1
Z[1] ← L ⊕ R
for i ← 2 to m do Z[i] ← Z[i − 1] ⊕ L(nty(i))
for i ← 1 to m − 1 do
    M[i] ← DK(C[i] ⊕ Z[i]) ⊕ Z[i]
X[m] ← len(M[m]) ⊕ L(−1) ⊕ Z[m]
Y[m] ← EK(X[m])
M[m] ← (first len(C[m]) bits of Y[m]) ⊕ C[m]
Checksum ←
    M[1] ⊕ ... ⊕ M[m − 1] ⊕ C[m]0* ⊕ Y[m]
Tag' ← EK(Checksum ⊕ Z[m]) [first τ bits]
    
```

- **M for D_k is the ciphertext**
- **N must be the same as in E_k**



Cifratura a chiave pubblica



RSA Public-Key Encryption (Rivest, Shamir & Adleman of MIT in 1977)

- **Plaintext and ciphertext are treated as integers between 0 and $n-1$ for some n**
- Uses exponentiation of integers modulo a prime
- Encrypt: $C = M^e \bmod n$
- Decrypt: $M = C^d \bmod n = (M^e)^d \bmod n = M^{ed} \bmod n$
- **Both sender and receiver know values of n and e**
- Only receiver knows value of d
- Public-key encryption algorithm with
 - public key $PU = \{e, n\}$
 - private key $PR = \{d, n\}$



RSA Requirements

- To be satisfactory for public-key encryption, the following requirements must be met:
 1. It is possible to find values of e , d , n such that $M^{ed} \bmod n = M$ for all $M < n$
 2. It is relatively easy to calculate M^e and C^d for all values of $M < n$
 3. It is infeasible to determine d given e and n
- **Req. 1 and 2 easily met**
- **Req. 3 can be satisfied for large values of e and n**



About RSA Req. 1: to find values of e, d, n such that $M^{ed} \bmod n = M$ for all $M < n$

- $M^{ed} \bmod n = M$ holds if e and d are multiplicative inverses modulo $\phi(n)$,
- $\phi(n)$ is the Euler totient function (*Funzione ϕ di Eulero o Toziente*)
 - is the number of positive integers less than n and relatively prime to n
 - It is proven that for p, q prime, $\phi(pq) = (p - 1)(q - 1)$
- The relationship between e and d can be expressed as
 - **$ed \bmod \phi(n) = 1 \rightarrow d \bmod \phi(n) = e^{-1}$**
- That is, e and d are multiplicative inverses mod $\phi(n)$.
- According to the rules of modular arithmetic, this is true only if
 - d (and therefore e) is relatively prime to $\phi(n)$, or
 - $\gcd(\phi(n), d) = 1$; gcd = greatest common divisor



Key Generation

Select p, q

p and q both prime, $p \neq q$

Calculate $n = p \times q$

Calculate $\phi(n) = (p - 1)(q - 1)$

Select integer e

$\gcd(\phi(n), e) = 1; 1 < e < \phi(n)$

Calculate d

$de \bmod \phi(n) = 1 \quad \gcd(\phi(n), d) = 1;$

Public key

$KU = \{e, n\}$

Private key

$KR = \{d, n\}$

Encryption

Plaintext:

$M < n$

Ciphertext:

$C = M^e \pmod n$

Decryption

Ciphertext:

C

Plaintext:

$M = C^d \pmod n$

RSA

Suppose A has published its public key and that B wishes to send the message M to A.

Then:

- B calculates $C = M^e \pmod n$ and transmits C

- A decrypts by calculating $M = C^d \pmod n$.

Example

- Computation of PU e PR
 - public key PU = {e, n}
 - private key PR = {d, n}
- Select two prime numbers
 - $p = 17, q = 11$.
- Calculate $n = p * q = 17 * 11 = 187$.
- Calculate $\phi(n) = (p - 1)(q - 1) = 16 * 10 = 160$.
- Select **e** such that
 - e is relatively prime to $\phi(n) = 160$
 - $e < \phi(n)$;
 - we choose $e = 7$.
- Determine **d** such that
 - $de \bmod 160 = 1$ and $d < 160$.
 - The correct value is $d = 23$, because $23 * 7 = 161 = (1 * 160) + 1$

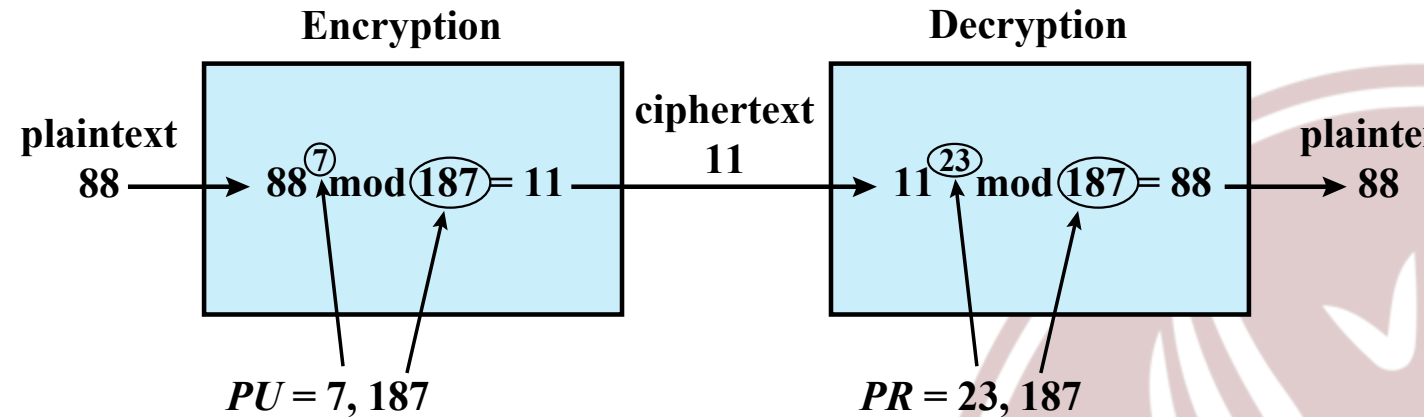


Figure 21.8 Example of RSA Algorithm

Security of RSA

Brute force

- Involves trying all possible private keys

Mathematical attacks

- There are several approaches, all equivalent in effort to factoring the product of two primes

Timing attacks

- These depend on the running time of the decryption algorithm

Chosen ciphertext attacks

- This type of attack exploits properties of the RSA algorithm



Mathematical Attacks to RSA

1. Factor n into its two prime factors. This enables calculation of $\phi(n) = (p - 1) * (q - 1)$, which, in turn, enables determination of $d \equiv e^{-1} \pmod{\phi(n)}$.
 2. Determine $\phi(n)$ directly, without first determining p and q . Again, this enables determination of $d \equiv e^{-1} \pmod{\phi(n)}$.
 3. Determine d directly, without first determining $\phi(n)$.
- Determining $\phi(n)$ given n is equivalent to factoring n (Attack 2)
 - With presently known algorithms, determining d given e and n appears to be at least as time consuming as the factoring problem (Attack 3)
 - Hence, we can use factoring performance as a benchmark against which to evaluate the security of RSA



The Factorization problem

- Factoring is an hard problem but not so hard!!!
- Until 1994 attack used the approach know as quadratic sieve
- From 1999 a new quadratic sieve algorithm used: Generalized Number Field Sieve (only 20% of computing effort needed before)
- Recommendations:
 - Key size 1024 – 2048
 - Selection of p and q to make factorization more complicated

Number of Decimal Digits	Number of Bits	Date Achieved
100	332	April 1991
110	365	April 1992
120	398	June 1993
129	428	April 1994
130	431	April 1996
140	465	February 1999
155	512	August 1999
160	530	April 2003
174	576	December 2003
200	663	May 2005
193	640	November 2005
232	768	December 2009

Recommendation on selection of p and q

1. p and q should differ in length by only a few digits
 1. for a 1024-bit key (309 decimal digits), both p and q should be on the order of magnitude of 10^{75} to 10^{100}
2. Both $(p - 1)$ and $(q - 1)$ should contain a large prime factor.
3. $\gcd(p - 1, q - 1)$ should be small

NOTE: it has been demonstrated that if $e < n$ and $d < n/4$, then d can be easily determined



Timing Attacks

- Paul Kocher, a cryptographic consultant, demonstrated that a snooper can determine a private key by keeping track of how long a computer takes to decipher messages
- Timing attacks are applicable not just to RSA, but also to other public-key cryptography systems
- This attack is alarming for two reasons:
 - It comes from a completely unexpected direction
 - It is a ciphertext-only attack



Timing Attack

- **Hypothesis:** the target system uses a modular multiplication function that is very fast in almost all cases but in a few cases takes much more time than an entire average modular exponentiation.
- The attack proceeds bit-by-bit starting with the leftmost bit, b_k .
- Therefore, if the observed time to execute the decryption algorithm is always slow when this particular iteration is slow with a 1 bit, then this bit is assumed to be 1. If a number of observed execution times for the entire algorithm are fast, then this bit is assumed to be 0.
- In practice,
 - modular exponentiation implementations do not have such extreme timing variations, in which the execution time of a single iteration can exceed the mean execution time of the entire algorithm.
 - Nevertheless, there is enough variation to make this attack practical.

Timing Attack Countermeasures

Constant exponentiation time

- Ensure that all exponentiations take the same amount of time before returning a result
- This is a simple fix but does degrade performance

Random delay

- Better performance could be achieved by adding a random delay to the exponentiation algorithm to confuse the timing attack
- If defenders do not add enough noise, attackers could still succeed by collecting additional measurements to compensate for the random delays

Blinding

- Multiply the ciphertext by a random number before performing exponentiation
- This process prevents the attacker from knowing what ciphertext bits are being processed inside the computer and therefore prevents the bit-by-bit analysis essential to the timing attack

Diffie-Hellman Key Exchange

- First published public-key algorithm
- By Diffie and Hellman in 1976 along with the exposition of public key concepts
- Used in a number of commercial products
- Practical method to exchange a secret key securely that can then be used for subsequent encryption of messages
- Security relies on difficulty of computing discrete logarithms



Global Public Elements

q	Prime number
α	$\alpha < q$ and α a primitive root of q

User A Key Generation

Select private X_A	$X_A < q$
Calculate public Y_A	$Y_A = \alpha^{X_A} \bmod q$

User B Key Generation

Select private X_B	$X_B < q$
Calculate public Y_B	$Y_B = \alpha^{X_B} \bmod q$

Generation of Secret Key by User A

$$K = (Y_B)^{X_A} \bmod q$$

Generation of Secret Key by User B

$$K = (Y_A)^{X_B} \bmod q$$

A and B have
exchanged a
secret value



Security of Diffie-Hellman

- because X_A and X_B are private, an adversary only has the following ingredients to work with:
- q , α , Y_A , and Y_B .
- The adversary is forced to take a discrete logarithm to determine the key. For example,
 - to determine the private key of user B, an adversary must compute: $X_B = d \log_{\alpha, q}(Y_B)$.
 - the adversary can then calculate the key K in the same manner as user B calculates it.
- The security of the Diffie-Hellman key exchange lies in the fact that, while it is relatively easy to calculate exponentials modulo a prime, it is very difficult to calculate discrete logarithms. For large primes, the latter task is considered infeasible.



Diffie-Hellman Example

Have

Prime number $q = 353$

Primitive root $\alpha = 3$

A and B each
compute their
public keys

A computes $Y_A = 3^{97} \bmod 353 = 40$

B computes $Y_B = 3^{233} \bmod 353 = 248$

Then exchange
and compute
secret key:

For A: $K = (Y_B)^{X_A} \bmod 353 = 248^{97} \bmod 353 = 160$

For B: $K = (Y_A)^{X_B} \bmod 353 = 40^{233} \bmod 353 = 160$

Attacker must
solve:

$3^a \bmod 353 = 40$ which is hard

Desired answer is 97, then compute key as B does



Diffie-Helman Key-exchange



Alice



Bob

Alice and Bob share a prime q and α , such that $\alpha < q$ and α is a primitive root of q

Alice generates a private key X_A such that $X_A < q$

Alice calculates a public key $Y_A = \alpha^{X_A} \bmod q$

Alice receives Bob's public key Y_B in plaintext

Alice calculates shared secret key $K = (Y_B)^{X_A} \bmod q$



Alice and Bob share a prime q and α , such that $\alpha < q$ and α is a primitive root of q

Bob generates a private key X_B such that $X_B < q$

Bob calculates a public key $Y_B = \alpha^{X_B} \bmod q$

Bob receives Alice's public key Y_A in plaintext

Bob calculates shared secret key $K = (Y_A)^{X_B} \bmod q$



Man-in-the-Middle Attack

- Attack is:
 1. Dath generates private keys XD1 and XD2, and their public keys YD1 and YD2
 2. Alice transmits YA to Bob
 3. Dath intercepts YA and transmits YD1 to Bob. Dath also calculates K2
 4. Bob receives YD1 and calculates K1
 5. Bob transmits YB to Alice
 6. Dath intercepts YB and transmits YD2 to Alice. Dath calculates K1
 7. Alice receives YD2 and calculates K2
- All subsequent communications compromised

Global Public Elements

q	Prime number
α	$\alpha < q$ and α a primitive root of q

User A Key Generation

Select private X_A	$X_A < q$
Calculate public Y_A	$Y_A = \alpha^{X_A} \bmod q$

User B Key Generation

Select private X_B	$X_B < q$
Calculate public Y_B	$Y_B = \alpha^{X_B} \bmod q$

Generation of Secret Key by User A

$$K = (Y_B)^{X_A} \bmod q$$

Generation of Secret Key by User B

$$K = (Y_A)^{X_B} \bmod q$$



Man-in-the-Middle Attack (cont'd)

- At this point, Bob and Alice think that they share a secret key, but instead Bob and Darth share secret key $K1$ and Alice and Darth share secret key $K2$. All future communication between Bob and Alice is compromised in the following way:
 1. Alice sends an encrypted message M : $E(K2, M)$.
 2. Darth intercepts the encrypted message and decrypts it, to recover M .
 3. Darth sends Bob $E(K1, M)$ or $E(K1, M')$, where M' is any message. In the first case, Darth simply wants to eavesdrop on the communication without altering it. In the second case, Darth wants to modify the message going to Bob.
- **The key exchange protocol is vulnerable to such an attack because it does not authenticate the participants. This vulnerability can be overcome with the use of digital signatures and public-key certificates.**



Digital signature applied to DE Key exchange

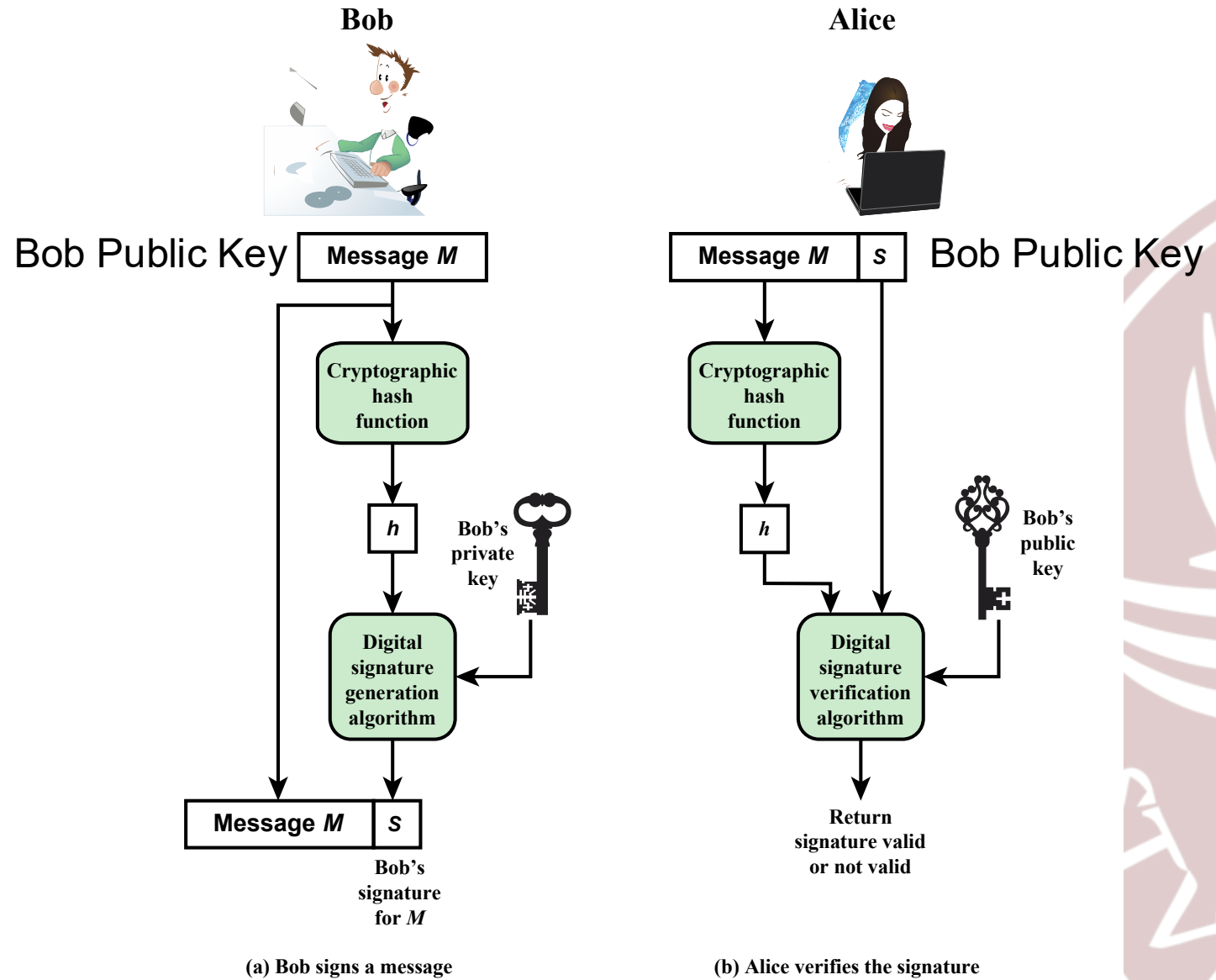


Figure 2.7 Simplified Depiction of Essential Elements of Digital Signature Process

Summary

- Secure hash functions
 - Simple hash functions
 - The SHA secure hash function
 - SHA-3
- Diffie-Hellman and other asymmetric algorithms
 - Diffie-Hellman key exchange
 - Other public-key cryptography algorithms
- Authenticated encryption
- The RSA public-key encryption algorithm
 - Description of the algorithm
 - The security of RSA
- HMAC
 - HMAC design objectives
 - HMAC algorithm
 - Security of HMAC